Introduction to Lab:

Phases of a Compiler:

A compiler is a system software (or set of programs) that translates text written in a computer language (the source language) into another computer language (the target language). The original sequence is usually called the source code and the output called object code. Commonly the output has a form suitable for processing by other programs (e.g., a linker), but it may be a human-readable text file. There are six (6) Phases of a Compiler and these are:

1. Lexical Analyzer: converts a sequence of characters into a sequence of tokens. Programs performing lexical analysis are called lexical analyzers or lexers. A lexer is often organized as separate scanner and tokenizer functions, though the boundaries may not be clearly defined.

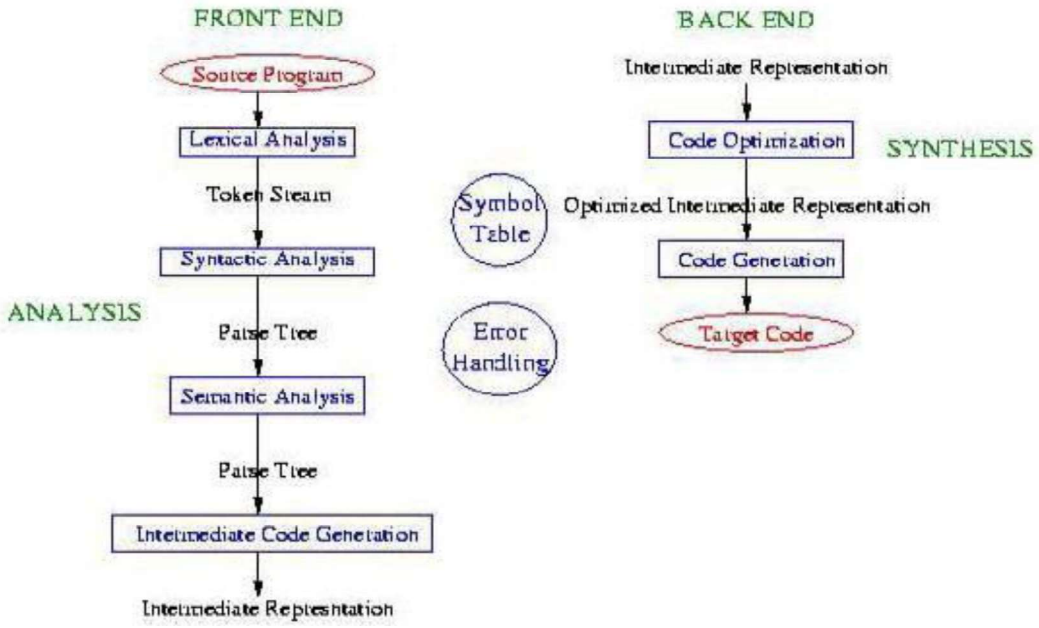
2. Syntax Analyzer: checks that something conforms to the rules of a given syntax and analysing its structure according to those rules.

3. Semantic Analyzer: checks the source program for semantic errors and gathers type information for the subsequent code-generation phase.

4. Intermediate Code Generator: is generated as a program for an abstract machine. It helps to easily translate into the target program. We consider the three-address code, similar to assembly.

5. Code Optimizer: attempts to improve the intermediate code, so that faster-running machine code will result.

6. Code Generator: is the process by which a compiler's code generator converts some internal representation of source code into a form (e.g., machine code) that can be readily executed by a machine (often a computer).



Lexical Analysis:

Token:

It is defined by two elements

1.token name

2.token attribute

A token is a name of a family of lexemes which follow a particular pattern.A token is signified by an abstract symbol or string to name that family.

Lexeme: A lexeme is an instance of a token family based on somerules of forming that token.

Pattern: A set of rules to form a given lexeme belonging to a token family.

Applications of Regular expression:

Regular expressions are useful in the production of Syntax highlighting systems, data validations, and many other tasks.

While regular expressions would be useful on search engine such as Google, processing them across the entire database could consume excessive computer resources depending on the complexity and design of the regex. Although in many cases system administrators

can run regex-based queries internally, most search engines do not offer regex support to the public.

Lexical analyzer realize,

1) type:

   string (to be carried out lexical analysis of source code),

   output: from (other kinds of code, its own value) which consists of binary sequences.

2) Function:

a. filter

b. Identification of spaces reserved words

c. hard

d. identifier to identify a number of

e. hard compound word

Parsing:

Syntax analysis *or* parsing is about discovering structure in text and is used to determine whether or not a text confirms to an expected format.

Top-down parsing:

Top-down parsing attempts to find the left-most derivations for an input string w, which is equivalent to constructing a parse tree for the input string w that starts from the root and creates the nodes of the parse tree in a predefined order. The reason that top-down parsing seeks the left-most derivations for an input string w and not the right-most derivations is that the input string w is scanned by the parser from left to right, one symbol/token at a time, and the left-most derivations generate the leaves of the parse tree in left-to-right order, which matches the input scan order.

Since top-down parsing attempts to find the left-most derivations for an input string *w*, a top-down parser may require backtracking (i.e., repeated scanning of the input); because in the attempt to obtain the left-most derivation of the input string *w*, a parser may encounter a situation in which a nonterminal *A* is required to be derived next, and there are multiple *A*-productions, such as $A \rightarrow \alpha_1 \mid \alpha_2 \mid \ldots \mid \alpha_n$.

In such a situation, deciding which *A*-production to use for the derivation of *A* is a problem. Therefore, the parser will select one of the *A*-productions to derive *A*, and if this

derivation finally leads to the derivation of *w*, then the parser announces the successful completion of parsing. Otherwise, the parser resets the input pointer to where it was when the nonterminal *A* was derived, and it tries another *A*-production. The parser will continue this until it either announces the successful completion of the parsing or reports failure after trying all of the alternatives.

Bottom-up parsing:

Bottom-up parsing is any method of discerning the hierarchical structure and meaning of a linear input text, by identifying and processing its lowest-level small details first, before its mid-level structures, and leaving the highest level over all structure to last.

Type of bottom-up parsers:

It is common for bottom-up parsers to take the form of general parsing engines, which can either parse, or generate a parser for, a specific programming language given a specification of its grammar. Perhaps the most well known generalized parser generators are YACC and GNU bison.

The common classes of bottom-up parsers are:

- LR parser
    - LR(0) - No lookahead symbol
    - SLR(1) - Simple with one lookahead symbol. Used by XPL.
    - LALR(1) - Lookahead bottom up, not as powerful as full LR(1) but simpler to implement. YACC deals with this kind of grammar.
    - LR(1) - Most general grammar, but most complex to implement.
    - LR(n) - (where n is a positive integer) indicates an LR parser with n lookahead symbols; while grammars can be designed that require more than 1 lookahead, practical grammars try to avoid this because increasing n can theoretically require exponentially more code and data space (in practice, this may not be as bad). Also, the class of LR(n) languages is the same as that of LR(1) languages.
- Precedence parsers
    - Simple precedence parser
    - Operator-precedence parser
    - Extended precedence parser

- Mixed Strategy Precedence parser, used by the original version of XPL.

Bottom-up parsing can also be done by backtracking.

Derivation:

Derivation from S means generation of string w from S. For constructing derivation two things are important:

i) Choice of non terminal from several others.

ii) Choice of rule from production rules for corresponding non terminal.Instead of choosing the arbitrary non terminal one can choose:

i) Either leftmost derivation – leftmost non terminal in a sentinel form

ii) or rightmost derivation – rightmost non terminal in a sentinel form

Parse tree:

A parse tree or it is also called concrete syntax tree that is ordered, rooted that represents the syntactic structure of a string. It is also a grammatical structure represented as a tree data structure.

Table-Based Parsing

A more flexible way of implementing a parser is to make it table-based, where we introduce a parsing table (a 2d array) and a stack. Each entry in the parse table represents a reduce action, and each entry on the stack is a symbol.

Here's a simple run-down of a table-based LL(1) parsing algorithm:

The bottom of the stack starts with **$** and the grammar's start symbol is on top of that. Look at the symbol on the top of stack, we'll call that **X**, and then look at the current token of the input, we'll call that **a**. If **X** is a terminal and it equals **a**, pop **X** off the stack and remove **a** from the input. If it doesn't equal **a**, an occur has occurred. If **X** is a non-terminal, retrieve the reduction from the table and swap it with **X**. **Table(X, a)** returns the string that **X** is reduced to when **a** is the next token in the input. That string is one of the alternatives on the right side of the production.

The string of symbols are actually thrown on the stack backwards since we're doing a leftmost derivation. If there's no entry in the table for **X** and **a** , an error has occurred. Repeat the steps until **X** equals **$**, where there are no more stack symbols to be processed.

That's literally the entire algorithm. If that example was difficult to understand, here's another example showing how the algorithm works.So how do we compute the parse table? You have to construct the **first** and **follow** sets. The input to these algorithms are a CFG. You can find the algorithms here. They aren't too complicated.

The important thing to know is that the parsing algorithm utilizes a table that's dynamically generated from the output of the first and follow sets, which can be programmatically generated from a grammar specification**.** No more hard-coding your grammar rules! And no more having to write code to support more language syntax! This gives rise to what we call the parser generator. This is the sane way to create a complex parser.

Introduction to Lex:

Lex is a unix utility which generates the lexical analyzer. The lex specification lex.l is given to Lex compiler to produce lex.yy.c. This lex.yy.c is a C program and consists of the tabular representation of the transition diagrams constructed for the regular expressions. The lex.yy.c is given to C compiler to produce object program a.out. When some input stream is given to a.out then sequence of tokens get generated. Lex specification consists of three parts:-

1) Declaration section 2) Translation rules 3) Auxiliary procedures %{

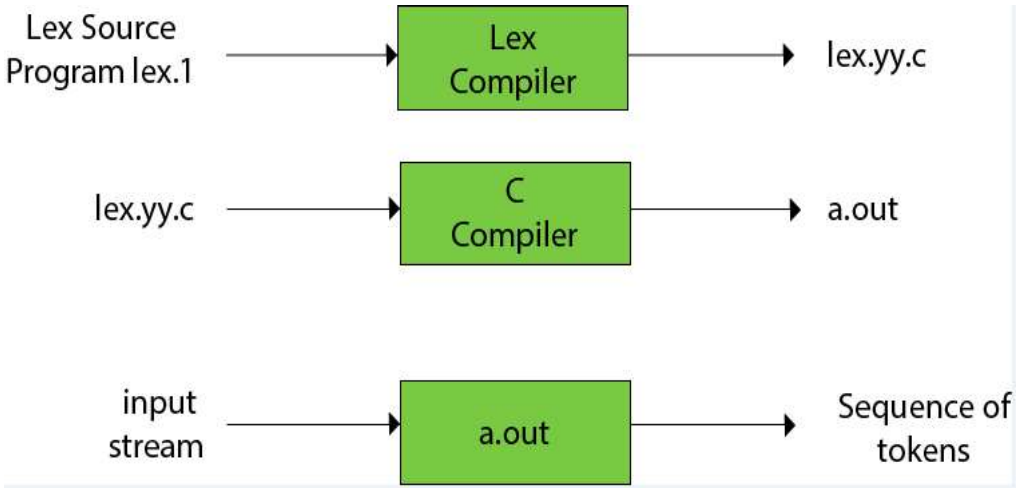
Declaration section %}

%%

Translation rules

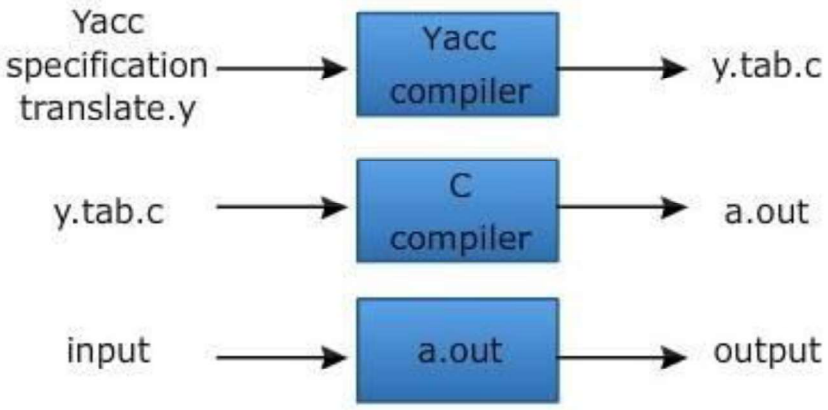
%%

Auxiliary procedures



Introduction to YACC

YACC (Yet Another Compiler Compiler) is a tool used to generate a parser. YACC translates a given Context Free Grammar (CFG) specifications (input in input_file.y) into a C implementation (y.tab.c) of a corresponding push down automaton (i.e., a finite state machine with a stack). This C program when compiled, yields an executable parser.



Lab Programs with Viva Questions:

1.Implementation Of Symbol Table Using C (With Output)

AIM:
To write a program for implementing Symbol Table using C.

ALGORITHM:

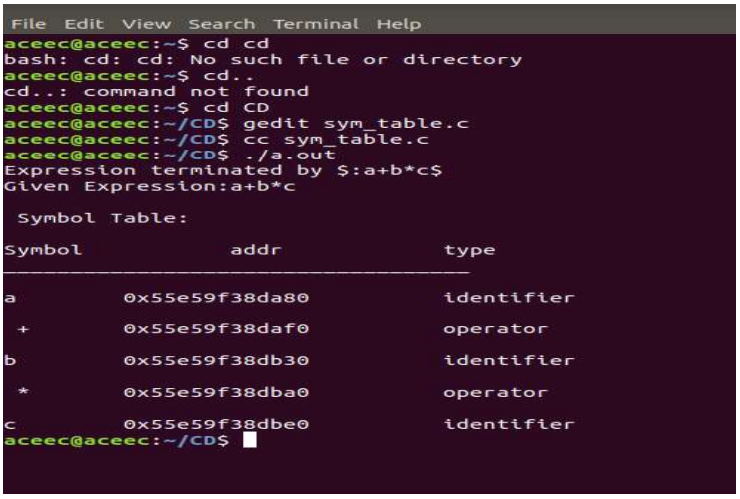
Step1: Start the program for performing insert, display, delete, search and modify option in symbol table
Step2: Define the structure of the Symbol Table
Step3: Enter the choice for performing the operations in the symbol Table
Step4: If the entered choice is 1, search the symbol table for the symbol to be inserted. If the symbol is already present, it displays "Duplicate Symbol". Else, insert the symbol and the corresponding address in the symbol table.
Step5: If the entered choice is 2, the symbols present in the symbol table are displayed.
Step6: If the entered choice is 3, the symbol to be deleted is searched in the symbol table.
Step7: If it is not found in the symbol table it displays "Label Not found". Else, the symbol is deleted.
Step8: If the entered choice is 5, the symbol to be modified is searched in the symbol table.

PROGRAM CODE:

```
//Implementation of symbol table
#include<stdio.h>
#include<ctype.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
void main()
{
int i=0,j=0,x=0,n;
void *p,*add[5];
char ch,srch,b[15],d[15],c;
printf("Expression terminated by $:");
while((c=getchar())!='$')
{
b[i]=c;
i++;
}
n=i-1;
printf("Given Expression:");
i=0;
while(i<=n)
{
```

```
printf("%c",b[i]);
i++;
}
printf("\n\n Symbol Table:\n");
printf("Symbol \t\t addr \t\t type\n");
printf("-------------------------------------------------------\n");
while(j<=n)
{
c=b[j];
if(isalpha(toascii(c)))
{
p=malloc(c);
add[x]=p;
d[x]=c;
printf("\n%c \t %p \t identifier\n",c,p);
x++;
j++;
}
else
{
ch=c;
f(ch=='+'||ch=='-'||ch=='*'||ch=='=')
{
p=malloc(ch);
add[x]=p;
d[x]=ch;
printf("\n %c \t %p \t operator\n",ch,p);
x++;
j++;
  }
 }
 }
}
```

OUTPUT:



2. Develop a Lexical Analyzer to recognize a few patterns inc.(Identifiers, Constants, Comments, Operators etc.))

```
/*lex code to determine whether input is an identifier or not*/
%{
#include <stdio.h>
%}

%%

“+”   {printf(“Addition operator”);}
“-”  {printf(“Subtraction operator”);}
“*”  {printf(“Multiplication operator”);}
“/”  {printf(“division operator”);}
“%”  {printf(“Modules operator”);}

/* identifying Comments*/
“//”.* {printf(“single Line Comment”);}
“/*”[a-zA-Z0-9]* “*/”  {printf(“Multi Line Comment”);}
/*  regex for Constants */
[0-9]+     {printf(“ Integer Constant”);}
[0-9]+.[0-9]+ {printf(“Float Constant”);}
[a-z]|[A-Z] {printf(“ character Constant”);}
[a-zA-Z0-9]+ {printf(“String Constant”);}

/* regex for valid identifiers */
^[a - z A - Z _][a - z A - Z 0 - 9 _] * printf("Valid Identifier");

/* regex for invalid identifiers*/
^[^a - z A - Z _] printf("Invalid Identifier");
.* {printf(“Invalid Identifier”);}

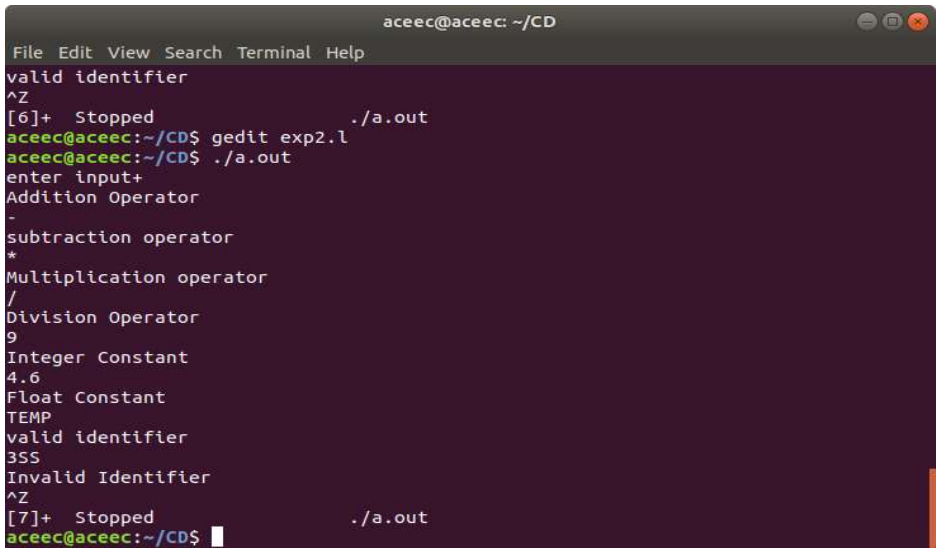
%%

int main()
{
```

```
Printf("enter input:");
yylex();
return 0;
}

Void yywrap()
{
return 1;
}
```

OUTPUT:



3. Implementation Of A Lexical Analyzer Using Lex (With Output)

AIM:

To write a program for implementing a Lexical analyser using LEX tool in Linux platform.

ALGORITHM:

Step1: In declaration section define COMMENT variable as 0.

Step2: Assign Name for Identifier pattern

Step3: start the rule section.

Step4: define pattern for keywords

Step 5: define pattern for Comment begin

Step 6: define pattern for BLOCK Begin

Step7: define pattern for BLOCK End

Step8:define pattern for NUMBER

Step9: define pattern for parenthesis

Step10: pattern for Assignment operator

Step 11: pattern for semicolon

Step12:pattern for relational operators

Step13: pattern for Preprocessor.

Step 14 : end of the rule section

Step 15: define main () function

Step 16: define yywrap() function.

Program Code:

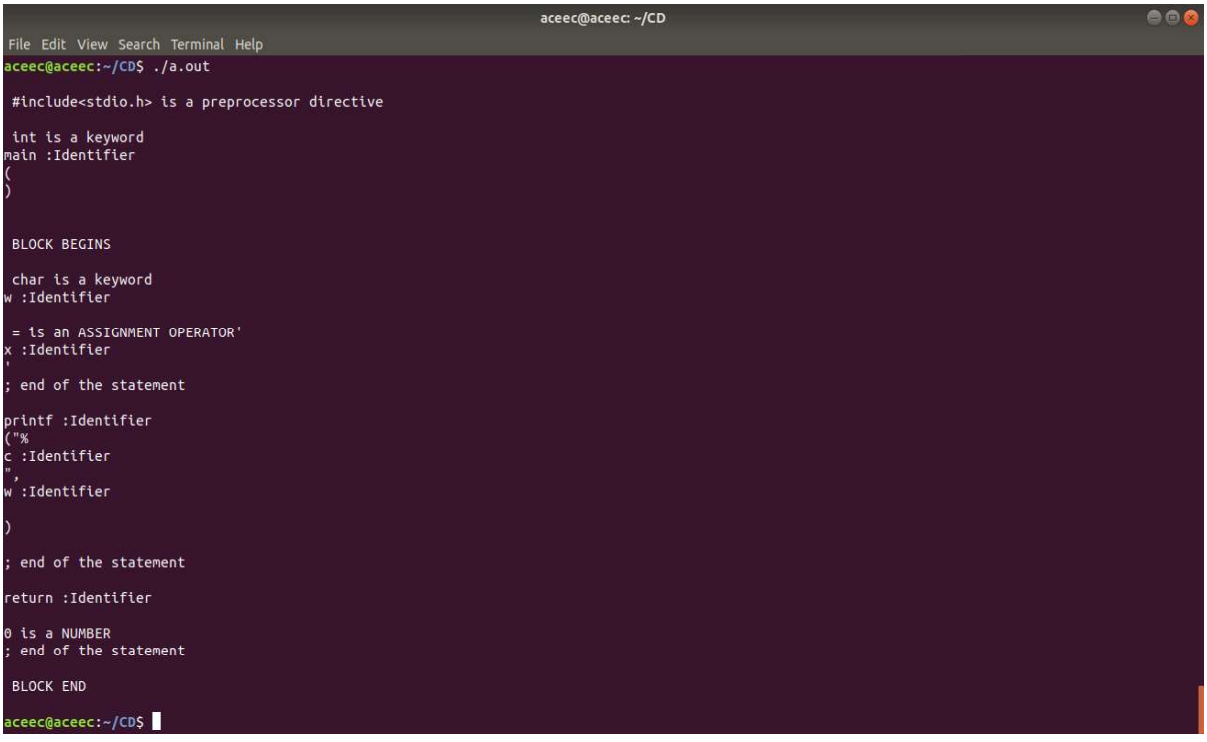
```
/* implementation of Lexical Analyzer using Lex Tool*/
%{
int COMMENT=0;
%}
Identifier [a-zA-Z][a-zA-Z0-9]*
%%
int|float|char|double|while|for|struct|typedef|do|if {printf("\n %s is a keyword",yytext);}
"/*" {COMMENT=1;printf("\n\t %s is a COMMENT",yytext);}
{Identifier}  {if(!COMMENT) printf("\n%s :Identifier\n",yytext);}
\{  {if(!COMMENT) printf("\n BLOCK BEGINS");}
\}  {if(!COMMENT) printf("\n BLOCK END");}
[0-9]+ {if(!COMMENT) printf("\n%s is a NUMBER",yytext);}
\)(\:)? {if(!COMMENT) printf("\n"); ECHO; printf("\n");}
\(ECHO;
= {if(!COMMENT) printf("\n %s is an ASSIGNMENT OPERATOR",yytext);}
";" {printf("\n%s end of the statement",yytext);}
\<=|\>=|\<|==|\>  {if(!COMMENT) printf("\n \t %s is a RELATIONAL OPERATOR",yytext);}
#.* {printf("\n %s is a preprocessor directive",yytext);}
%%
int main(int argc,char **argv)
{
FILE *fp;
fp=fopen("wel.c","r");
if(!fp)
{
```

```
printf("cloud not open the fle");
exit(0);
}
yyin=fp;
yylex();
printf("\n");
return 0;
}
int yywrap()
{
return 1;
}
```

INPUT:

```
//var.c
#include<stdio.h>
void main()
{
int a,b,c;
a=1;b=2;
c=a+b;
Printf("sum: %d",c);
}
```

OUT PUT:



4. Generate yacc specifications for a few syntactic categories.

a) program to recognize a valid arithmetic expression that uses operator +,-,*and /.

Program name : arith_exp.l

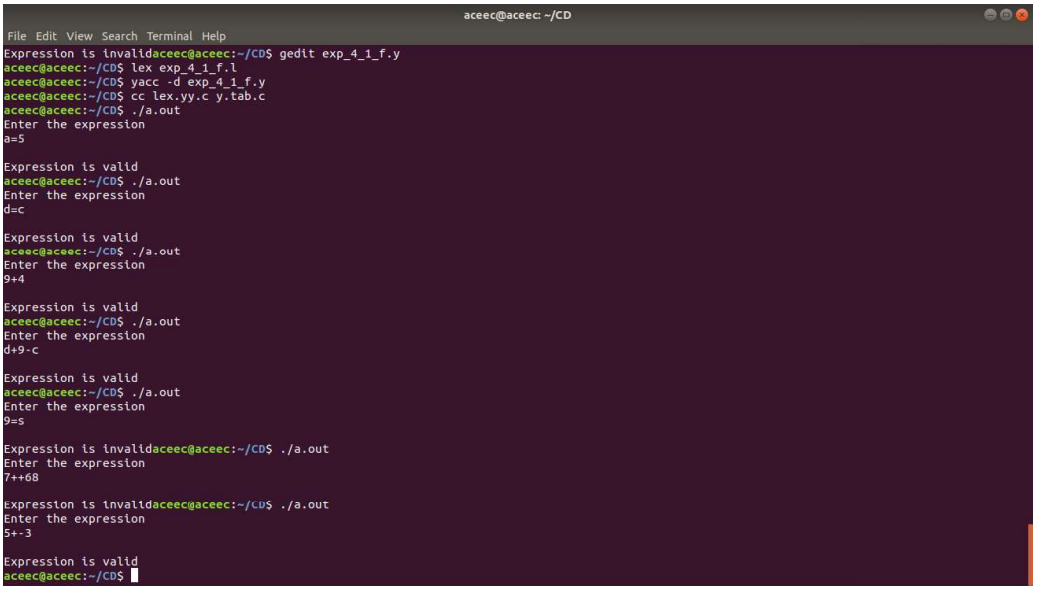
```
%{
#include"y.tab.h"
extern int yylval;
%}
/* defined section */
%%
[0-9]+ {yylval=atoi(yytext); return NUMBER;}
[a-zA-Z]+ {return ID;}
[\t]+ ;
\n {return 0;}
. {return yytext[0];}
%%
int yywrap()
{
return 1;
}
```

Program Name : arith_exp.y

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex();
int yyerror(char *);
%}
//definition section
%token NUMBER ID
%left '+' '-'
%left '*' '/'
%%
S:A'='expr|expr
;
expr: expr '+' expr
|expr '-' expr
|expr '*' expr
|expr '/' expr
|'-' NUMBER
|'-' ID
|'('expr')'
|NUMBER
|ID
;
A:ID
;
%%
//main function
int main()
{
printf("Enter the expression\n");
yyparse();
printf("\nExpression is valid\n");
return 0;
}
```

```
//if error occured
Int  yyerror(char *s)
{
printf("\nExpression is invalid");
exit(0);
}
```

Output:



b) program to recognize a valid variable which starts with a letter followed by any number of letters or digits.

Program name: var_test.l

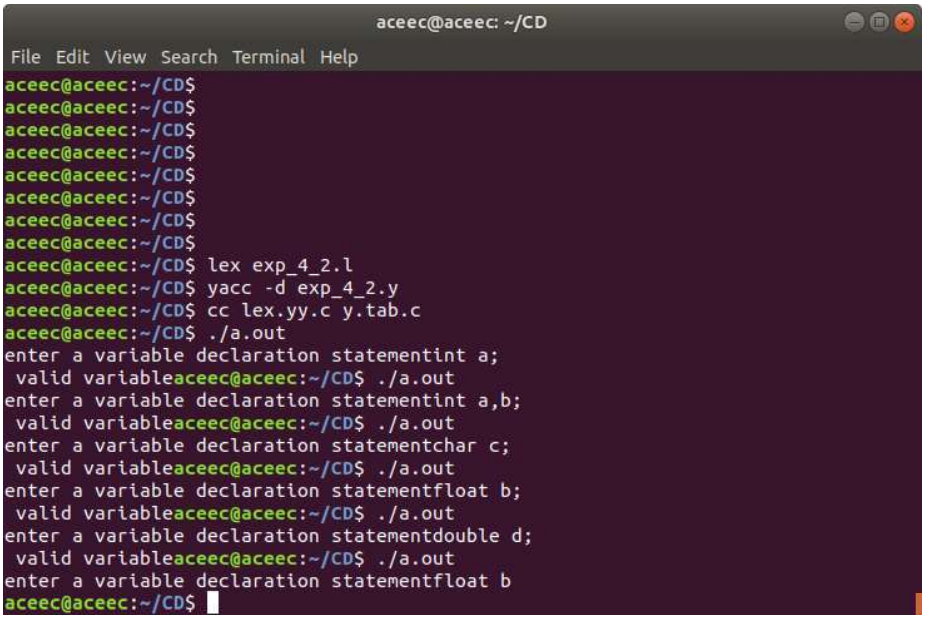
```
%{
#include "y.tab.h"
%}
%%
int    {return INT;}
float  {return FLOAT;}
double {return DOUBLE;}
char   {return CHAR;}
[a-zA-Z]+ {return ID;}
";" {return SEM;}
"," {return COM;}
\n return 0;
%%
int yywrap()
{
return 1;
}
```

Program Name :var_test.y

```
%{
#include<stdio.h>
int yylex();
int yyerror(char*);
%}
%token ID INT FLOAT DOUBLE CHAR SEM COM
%%
```

```
D:T L SEM{ printf("valid variable");}
;
L:L COM ID
|ID
;
T:INT
|FLOAT
|DOUBLE
|CHAR
;
%%
void main()
{
printf("enter a variable declaration statement");
yyparse();
}
int yyerror(char *s)
{
printf("Invalid declaration");
return 0;
}
```

Out put:



c) implementation of calculator using Lex and Yacc.

Lex code: cal.lex

```
%{
#include <stdlib.h>
#include "y.tab.h"
%}
%%
[0-9]+  {yylval = atoi(yytext); return INTEGER;}
[-+\n]   return *yytext;
[ \t]      ; /* skip whitespace */
%%
int yywrap(void)
```

```
{
return 1;
}
```

Yacc code: cal.yacc

```
%{
#include <stdio.h>
int yylex(void);
void yyerror(char *);
%}
%token INTEGER
%left  '+' '-'
%left  '*' '/' '%'
%%
program:
program expr '\n'         { printf("result:%d\n", $2); }
|
;
expr: INTEGER                 { $$ = $1; }
      | expr '+' expr         { $$ = $1 + $3; }
      | expr '-' expr         { $$ = $1 - $3; }
      |expr '*' expr {$$=$1 *$3}
      |expr '/' expr {$$=$1 / $3}
      |expr '%' expr {$$=$1%$3}
       ;
%%
void yyerror(char *s)
{ fprintf(stderr, "%s\n", s);
}
int main(void)
{
printf("enter an Expression:");
yyparse();
return 0;
}
```

Input:
Enter an Expression:5+8
Out put:
13.

5.Program to Convert the BNF rules into Yacc form and write code to generate Abstract Syntax Tree

ALGORITHM:

1. Start

2. Include the header file.

3. In int code.l,declare the variable lie no as integer and assign it to be equal to ‘1’.

4. Start the int code.l with declarative section.

5. In translation rules section define keywords ,data types and integer along with their actions .

6. Start the main block. In main block check the statement

7.

1.declarative

2.assignment

3.conditional

4.if and else

5.While assignment.

8. Perform the actions of that particular block.

9. In main program declare the parameters arg c as int end *argv[] as char.

10. In main program open file in read mode.

11. Print the output in a file.

12. End

Lex <BNF>

```
%{
#include"y.tab.h"
#include<stdio.h>
#include<string.h>
int LineNo=1;
%}
identifier [a-zA-Z][_a-zA-Z0-9]*
```

```
number [0-9]+|([0-9]*\.[0-9]+)

%%

main\(\) return MAIN;

if return IF;

else return ELSE;

while return WHILE;

int |char |float   {return TYPE;}

{identifier}  {strcpy(yylval.var,yytext); return VAR;}

{number}  {strcpy(yylval.var,yytext); return NUM;}

\< |\> |\>= |\<= |==  {strcpy(yylval.var,yytext); return RELOP;}

[ \t] ;

\n  LineNo++;

.*   return yytext[0];

%%
```

Yacc Code:

```
%{

#include<stdio.h>

#include<stdlib.h>

struct quad

{

char op[5];

char arg1[10];

char arg2[10];

char result[10];

}QUAD[30];

struct stack
```

```
{
int items[100];
int top;
}stk;
int Index=0,tIndex=0,StNo,Ind,tInd;
extern int LineNo;
void yyerror();
int  yylex();
%}
%union
{
char var[10];
}
%token  NUM VAR RELOP
%token MAIN IF ELSE WHILE TYPE
%type  EXPR ASSIGNMENT CONDITION IFST ELSEST WHILELOOP
%left '-'+'
%left '*'/'
%%
PROGRAM : MAIN BLOCK
;
BLOCK: '{' CODE '}'
;
CODE: BLOCK
| STATEMENT CODE
| STATEMENT
```

```
;
STATEMENT: DESCT ';'
| ASSIGNMENT ';'
| CONDST
| WHILEST
;
DESCT: TYPE VARLIST
;
VARLIST: VAR ',' VARLIST
| VAR
;
ASSIGNMENT: VAR '=' EXPR{
strcpy(QUAD[Index].op,"=");
strcpy(QUAD[Index].arg1,$3);
strcpy(QUAD[Index].arg2,"");
strcpy(QUAD[Index].result,$1);
strcpy($$,QUAD[Index++].result);
};
EXPR: EXPR '+' EXPR {AddQuadruple("+",$1,$3,$$);}
| EXPR '-' EXPR {AddQuadruple("-",$1,$3,$$);}
| EXPR '*' EXPR {AddQuadruple("*",$1,$3,$$);}
| EXPR '/' EXPR {AddQuadruple("/",$1,$3,$$);}
| '-' EXPR {AddQuadruple("UMIN",$2,"",$$);}
| '(' EXPR ')' {strcpy($$,$2);}
| VAR
| NUM
```

```
;
CONDST: IFST{
Ind=pop();
sprintf(QUAD[Ind].result,"%d",Index);
Ind=pop();
sprintf(QUAD[Ind].result,"%d",Index);
}
| IFST ELSEST
;
IFST: IF '(' CONDITION ')' {
strcpy(QUAD[Index].op,"==");
strcpy(QUAD[Index].arg1,$3);
strcpy(QUAD[Index].arg2,"FALSE");
strcpy(QUAD[Index].result,"-1");
push(Index);
Index++;
}
BLOCK {
strcpy(QUAD[Index].op,"GOTO");
strcpy(QUAD[Index].arg1,"");
strcpy(QUAD[Index].arg2,"");
strcpy(QUAD[Index].result,"-1");
push(Index);
Index++;
};
ELSEST: ELSE{
```

```
tInd=pop();
Ind=pop();
push(tInd);
sprintf(QUAD[Ind].result,"%d",Index);
}
BLOCK{
Ind=pop();
sprintf(QUAD[Ind].result,"%d",Index);
};
CONDITION: VAR RELOP VAR {AddQuadruple($2,$1,$3,$$);
StNo=Index-1;
}
| VAR
| NUM
;
WHILEST: WHILELOOP{
Ind=pop();
sprintf(QUAD[Ind].result,"%d",StNo);
Ind=pop();
sprintf(QUAD[Ind].result,"%d",Index);
};
WHILELOOP: WHILE '(' CONDITION ')' {
strcpy(QUAD[Index].op,"==");
strcpy(QUAD[Index].arg1,$3);
strcpy(QUAD[Index].arg2,"FALSE");
strcpy(QUAD[Index].result,"-1");
```

```
push(Index);
Index++;
}
BLOCK {
strcpy(QUAD[Index].op,"GOTO");
strcpy(QUAD[Index].arg1,"");
strcpy(QUAD[Index].arg2,"");
strcpy(QUAD[Index].result,"-1");
push(Index);
Index++;
};
%%
extern FILE *yyin;
int main(int argc,char *argv[])
{
FILE *fp;
int i;
if(argc>1)
{
fp=fopen(argv[1],"r");
if(!fp)
{
printf("\n File not found");
exit(0);
}
yyin=fp;
```

```
}
yyparse();
printf("\n\n\t\t ----------\n\t\t Pos Operator Arg1 Arg2 Result\n\t\t----------");
for(i=0;i
{
printf("\n\t\t %d\t %s\t %s\t %s\t %s",i,QUAD[i].op,QUAD[i].arg1,QUAD[i].arg2,QUAD[i].result);
}
printf("\n\t\t ----------");
printf("\n\n");
return 0;
}
void push(int data)
{stk.top++;
if(stk.top==100)
{
printf("\n Stack overflow\n");
exit(0);
}stk.items[stk.top]=data;
} int pop()
{ int data;
if(stk.top==-1)
{
printf("\n Stack underflow\n");
exit(0);
}
data=stk.items[stk.top--];
```

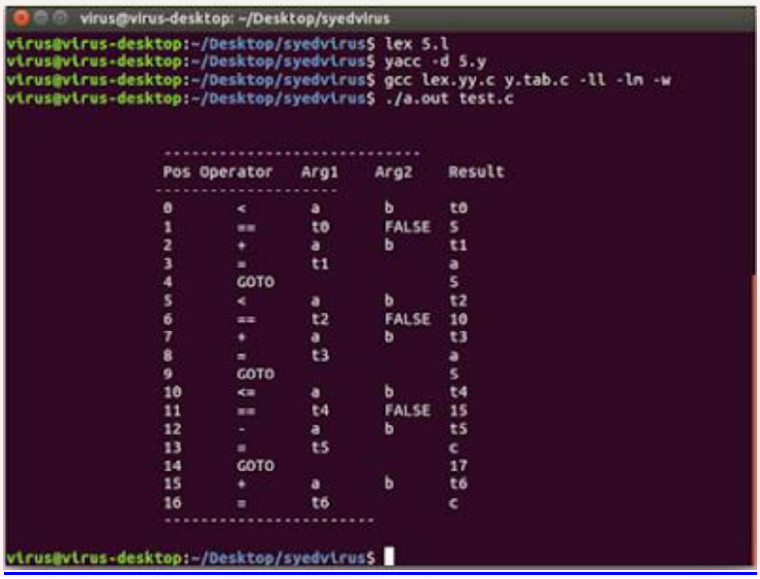
```
return data;
}
void AddQuadruple(char op[5],char arg1[10],char arg2[10],char result[10])
{strcpy(QUAD[Index].op,op);
strcpy(QUAD[Index].arg1,arg1);
strcpy(QUAD[Index].arg2,arg2);
sprintf(QUAD[Index].result,"t%d",tIndex++);
strcpy(result,QUAD[Index++].result);
}
void yyerror()
{
printf("\n Error on line no:%d",LineNo);
}
```

Input

```
main()
{ int a,b,c;
if(a<b)
{a=a+b;}

while(a<b)
{a=a+b;}
if(a<=b)
{c=a-b;}
else
{c=a+b;}
}
```

Out put:



6. Implement Type Checking.

Aim:

to write a C program to implement type checking.

Algorithm:

Step-1: Track the global scope type information.

Step-2: Determine the type of expression.

Step-3: if type found correct, do the operation

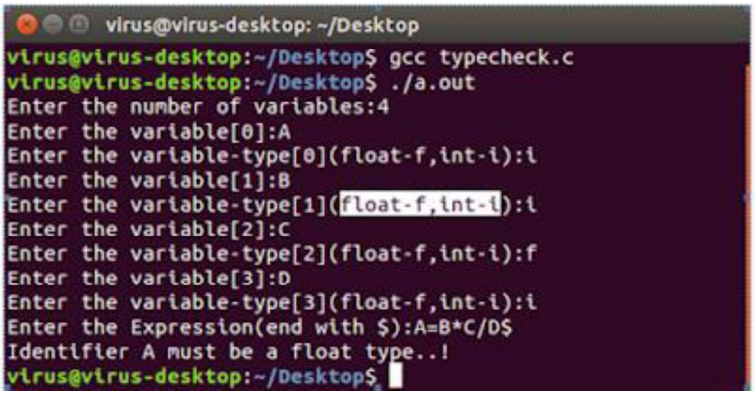
Step-4: Type mismatches, semantic error will be notified.

Program Code:

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
int n,i,k,flag=0;
char var[15],type[15],b[15],c;
printf("enter the no.of variables");
scanf("%d",&n);
for(i=0;i<n;i++)
{
flushall();
printf("enter the variable[%d]:",i);
scanf("%c",&var[i]);
flushall();
printf("Enter the variable -type[%d](float-f, int-i):",i);
scanf("%c",&type[i]);
if(type[i]=='f')
```

```
flag=1;
}
printf("enter the expression(end with $):");
i=0;
getchar();
while((c=getchar())!='$')
{
b[i]=c;
i++;
}
k=i;
for(i=0;i<k;i++)
{
if (b[i]=='/')
{
flag=1;
break;
}
}
for (i=0;i<n;i++)
{
if(b[0]==var[i])
{
if(flag==1)
{
if (type[i]=='f')
{
printf("\nthe datatype is correctly defined…..!\n");
break;
}
else
{
printf("identifier %c must be a float type…..!\n",var[i]);
break;
}
}
else
{
 printf("\n the data type is correctly defined….!\n");
break;
}
}
}
return 0;
}
```

OUT PUT:



7. Implement Any One Storage Allocation Strategies(Heap,Stack,Static)

AIM: To implement Heap storage allocation strategy using C program.

Algorithm:

Step-1: define node structure for Heap storage allocation

Step-2: perform operations on Heap storage:

i) Create operation

ii) Insert operation:

a) Insert at Head

b) Insert at end

c) Insert at after existed node

iii) Search operation

iv) Deletion a node from Heap storage

Program Code:

```
#include<stdio.h>
#include<stdlib.h>
#define TRUE 1
#define FALSE 0
typedef struct Heap
{
 int data;
struct Heap *next;
} node;
node *head=NULL,*temp,*new1,*t;
void create();
void display();
node * search(int);
node* insert();
void  delete1();
void get_node();
int val;
char ans,ch;
int main()
{
int choice;
```

```
do
{
printf("\n program to perform various operations on heap using Dynamic memory management");
printf("\n 1.create");
printf("\n 2. Display");
printf("\n 3.insert an element in a list");
printf("\n 4. Delete an element from the list");
printf("\n 5. Quit\n");
printf("enter your choice (1-5):");
scanf("%d",&ch);
switch (ch)
{
case 1: create();break;
case 2: display();break;
case 3: head=insert();break;
case 4: delete1();break;
case 5: exit(0);
default:
printf("Invalid choice, try again");
}
}
while(choice!=5);
return 0;
}

void create()
{
do
{
printf("enter the element:");
scanf("%d",&val);
 get_node();
if(new1==NULL)
  printf("\n memory is not allocated");
new1->data=val;
if(head==NULL)
{
head=new1;
temp=head;
}
else
{
temp->next=new1;
temp=new1;
}
//flushall();
while(getchar()!='\n');
printf("\n do you want to enter more elements?(y/n):");
//ans=getchar();
scanf("%c",&ans);
}
while(ans=='y');
printf("\n the list is created\n");
```

```
}
void get_node()
{
new1=(node*)malloc(sizeof(node));
new1->next=NULL;
}
void display()
{
temp=head;
if (temp==NULL)
{
printf("\n the list is empty\n");
return ;
}
while(temp!=NULL)
{
printf("%d->",temp->data);
temp=temp->next;
}
printf("NULL");
}

node* search(int key)
{
int found;
temp=head;
if(temp==NULL)
{
printf("the list is empty\n");
return NULL;
}
found=FALSE;
while(temp!=NULL && found==FALSE)
{
if(temp->data!=key)
  temp=temp->next;
else
{
found=TRUE;
printf("\n the element is presented in the list");
return temp;
}
}
if (found==FALSE)
{
printf("\n the element is not presented in the list");
return NULL;
}
return NULL;
}
node* insert()
{
int choice;
```

```
node * insert_head();
void insert_after();
void insert_last();
printf("\n 1. Insert a node as a head node");
printf("\n 2. Insert a node as a Last node");
printf("\n 3. Insert a node at specified position in the list");
printf("\n enter your choice for insertion of a node:");
scanf("%d",&choice);
switch(choice)
{
case 1: head=insert_head(); break;
case 2: insert_last();break;
case 3: insert_after(); break;
}
return head;
}
node* insert_head()
{
get_node();
printf("\n enter the element ");
scanf("%d",&new1->data);
if(head==NULL)
head=new1;
else
{
temp=head;
new1->next=temp;
head=new1;
}
return head;
}

void insert_last()
{
get_node();
printf("\n enter the element which you want to insert");
scanf("%d",&new1->data);
if(head==NULL)
head=new1;
else
{
temp=head;
while(temp->next!=NULL)
temp=temp->next;
temp->next=new1;
new1->next=NULL;
}
}
void insert_after()
{
int key;
get_node();
printf("\n enter the element");
```

```
scanf("%d",&new1->data);
if(head==NULL)
head=new1;
else
{
printf("\n After which node new node insert");
scanf("%d",&key);
t=search(key);
if(t!=NULL)
{
new1->next=t->next;
t->next=new1;
}
else
printf("%d can not insert in LIst",new1->data);
}
}

node* get_prev(node *t)
{
node *prev;
temp=head;
if(temp==NULL)
return NULL;
while(temp!=t)
{
prev=temp;
temp=temp->next;
}
return prev;
}

void delete1()
{
node *prev;
int key;
if (head==NULL)
{
printf("\n the list is empty");
return;
}
printf("\n enter the element you want to delete");
scanf("%d",&key);
t=search(key);
if(t!=NULL)
{
prev=get_prev(t);
if(prev!=NULL)
{
prev->next=t->next;
free(t);
}
else
```

```
{
head=temp->next;
free(temp);
}
printf("\n the element is deleted");
}
}
```

Input & Out put:

```
program to perform various operations on heap using Dynamic memory management");
 1.create
2. Display
3.insert an element in a list
 4. Delete an element from the list
 5. Quit
enter your choice (1-5):1
enter a element :6
do you want more elements(y/n):y
enter a element:9
do you want more elements(y/n):n
1.create
2. Display
3.insert an element in a list
 4. Delete an element from the list
 5. Quit
enter your choice (1-5):2
elements in storage: 6->9->NULL
1.create
2. Display
3.insert an element in a list
 4. Delete an element from the list
 5. Quit
enter your choice (1-5):5
```

8. a) Lex Program to count the numbers of lines, words, spaces, and characters in a given statement

```
/*Lex Program to count numbers of lines, words, spaces and characters
in a given statement*/
%{
#include<stdio.h>
int sc=0,wc=0,lc=0,cc=0;
%}
%%
[\n] { lc++; cc+=yyleng;}
[  \t] { sc++; cc+=yyleng;}
[^\t\n ]+ { wc++;  cc+=yyleng;}
%%
int main(int argc ,char* argv[ ])
{
        printf("Enter the input:\n");
        yylex();
        printf("The number of lines=%d\n",lc);
        printf("The number of spaces=%d\n",sc);
```

```
        printf("The number of words=%d\n",wc);
        printf("The number of characters are=%d\n",cc);
}

int yywrap( )
{
        return 1;
}
```

Compiletion:

```
$ lex ch_count.l
$ gcc lex.yy.c
```

Execution: $ ./a.out

Input:

```
Enter the input: Greetings from IT Dept
Welcome to ACE Engineering College
Lex Yacc Tutorial
```

Out put:

```
The number of lines=3
The number of spaces=8
The number of words=12
The number of characters are=90
```

8.b)Program to count the numbers of lines, words, spaces, and characters from a given file

```
%{
#include<stdio.h>
int sc=0,wc=0,lc=0,cc=0;
%}
%%
[\n] { lc++; cc+=yyleng;}
[  \t] { sc++; cc+=yyleng;}
[^\t\n ]+ { wc++;  cc+=yyleng;}
%%
int main(int argc ,char* argv[])
{
        if(argc==2)
        {
                yyin=fopen(argv[1],"r");
        }
        else
        {
                printf("Enter the input\n");
                yyin=stdin;
        }
        yylex();
        printf("The number of lines=%d\n",lc);
        printf("The number of spaces=%d\n",sc);
        printf("The number of words=%d\n",wc);
        printf("The number of characters are=%d\n",cc);
}
int yywrap( )
{
        return 1;
```

```
}
```
Out put:

The number of lines=3

The number of spaces=8

The number of words=12

The number of characters are=90

9. Write A C-Program To Implement Lexical Analysis.

AIM : To write a program to implement Lexical Analysis using C.
ALGORITHM:
Step 1 : Start the program
Step 2 : Include necessary header files.
Step 3: The ctype header file is to load the file with predicate, isdigit.
Step 4 : The define directive defines the buffer size, numerics, assignment operator, relational operator.
Step 5 : Initialize the necessary variables.
Step 6: To return index of new string S, token t using insert() function.
Step 7 : Initialize the length of every string.
Step 8: Check the necessary condition.
Step 9: Call the initialize() function. This function loads the keywords into the symbol table.
Step 10 : Check the conditions such as white spaces, digits, letters and alphanumerics.
Step 11 : To return index of entry for string S, or 0 if S is not found using lookup( ) function.
Step 12 : Check this until EOF is found.
Step 13 : Otherwise initialize the token value to be none.
Step 14 : In the main function if lookahead equals numeric then the value of attribute num is given by the global variable tokenval.
Step 15 : Check the necessary conditions such as arithmetic operators , parenthesis , identifiers, assignment operators and relational operators.
Step 16 : Stop the program.

```
/*STANDALONE SCANNER PROGRAM*/
#include<stdio.h>
#include<ctype.h>
#include<string.h>
int main()
{
FILE *fp1,*fp2;
int l=1,t=0,j=0,i,flag;
char ch,str[20];
char keyword[30][30] = {"int","float","if","else","do","while"};
fp1=fopen("input.txt","r");
```

```
fp2=fopen("output.txt","w");
if(fp1==NULL)
{
 printf("file is not existed");
 exit(0);

}
fprintf(fp2,"Line no. \t Token no. \t Token \t Lexeme\n\n");
while(!feof(fp1))
{
i=0;
flag=0;
ch=fgetc(fp1);
if( ch=='+' || ch== '-' || ch=='*' || ch=='/' )
{
fprintf(fp2,"%7d\t\t %7d\t\t Operator\t %7c\n",l,t,ch);
t++;
}
else if( ch==';' || ch=='{' || ch=='}' || ch=='(' || ch==')' || ch=='?' ||
ch=='@' || ch=='!' ||
ch=='%')
{
fprintf(fp2,"%7d\t\t %7d\t\t Special symbol\t %7c\n",l,t,ch);
t++;
}
else if(isdigit(ch))
{
fprintf(fp2,"%7d\t\t %7d\t\t Digit\t\t %7c\n",l,t,ch);
t++;
}
else if(isalpha(ch))
{
str[i]=ch;
i++;
ch=fgetc(fp1);
while(isalnum(ch) && ch!=' ')
{
str[i]=ch;
i++;
ch=fgetc(fp1);
}
str[i]='\0';
for(j=0;j<=5;j++)
{
if(strcmp(str,keyword[j])==0)
{
flag=1;
break;
}
}
if(flag==1)
{
fprintf(fp2,"%7d\t\t %7d\t\t Keyword\t %7s\n",l,t,str);
```

```
t++;
}
else
{
fprintf(fp2,"%7d\t\t %7d\t\t Identifier\t %7s\n",l,t,str);
t++;
}
}
else if(ch=='\n')
{
l++;
}
else if(ch=='#')
{
fprintf(fp2,"%7d\t\t %7d\t\t preprossesor\t %7c\n",l,t,ch);
t++;
}
}
fclose(fp1);
fclose(fp2);
return 0;
}
```

Input:

```
//input.txt
#include<stdio.h>
void main()
{
printf("Hello World");
}
```

Output:

```
//output.txt
Line no. Token no. Token Lexeme
1          0        Identifier include
1          1        Identifier stdio
1          2        Identifier h
2          3        Identifier void
2          4        Keyword main
2          5        Special symbol )
3          6        Special symbol {
4          7        Identifier printf
4          8        Identifier Hello
4          9        Identifier World
4          10       Special symbol )
4          11       Special symbol ;
5          12       Special symbol }
```

10. write a recursive descent parser for the grammar:
E□E+T E□T T□T*F T□F F□(E)|id

Steps:

1. It repeatedly scans the ***input*** until we locate the right path.
2. The following qualities of the grammar must be present in order to create a recursive descent parser:
3. It shouldn't be allowed to repeat.

4. It needs to be ***left-factored***. Alternatives shouldn't share a prefix.
5. Recursion should be possible in languages.
6. Backtracking will be required by the recursive descent parser if the grammar is not left-factored.

Given Grammar:

Before removing left recursion	**After removing left recursion**
E->E+T|T T->T*F|F F ->(E) | id	E->TE' E->+TE'|e T->FT' T->*FT|e F ->(E) | id

Note: 'e' for epsilon.

```
#include <stdio.h>
#include <string.h>
#define SUCCESS 1
#define FAILED 0
int E(), Edash(), T(), Tdash(), F();
const char *cursor;
char string[64];
int main()
{
puts("Enter the string");
// scanf("%s", string);
sscanf("i+(i+i)*i", "%s", string);
cursor = string;
puts("");
puts("Input      Action");
puts("--------------------------------");
if (E() && *cursor == '\0') {
puts("--------------------------------");
puts("String is successfully parsed");
return 0;
} else {
puts("--------------------------------");
puts("Error in parsing String");
return 1;
}
}

int E()
{
printf("%-16s E -> T E'\n", cursor);
if (T()) {
if (Edash())
return SUCCESS;
else
return FAILED;
} else
```

```
return FAILED;
}

int Edash()
{
if (*cursor == '+') {
printf("%-16s E' -> + T E'\n", cursor);
cursor++;
if (T()) {
if (Edash())
return SUCCESS;
else
return FAILED;
} else
return FAILED;
} else {
printf("%-16s E' -> $\n", cursor);
return SUCCESS;
}
}

int T()
{
printf("%-16s T -> F T'\n", cursor);
if (F()) {
if (Tdash())
return SUCCESS;
else
return FAILED;
} else
return FAILED;
}

int Tdash()
{
if (*cursor == '*') {
printf("%-16s T' -> * F T'\n", cursor);
cursor++;
if (F()) {
if (Tdash())
return SUCCESS;
else
return FAILED;
} else
return FAILED;
} else {
printf("%-16s T' -> $\n", cursor);
return SUCCESS;
}
}

int F()
{
```

```
if (*cursor == '(') {
printf("%-16s F -> ( E )\n", cursor);
cursor++;
if (E()) {
if (*cursor == ')') {
cursor++;
return SUCCESS;
} else
return FAILED;
} else
return FAILED;
} else if (*cursor == 'i') {
cursor++;
printf("%-16s F ->i\n", cursor);
return SUCCESS;
} else
return FAILED;
}
```

Output:

```
Input              Action
--------------------------------
i+(i+i)*i       E -> T E'
i+(i+i)*i       T -> F T'
+(i+i)*i        F ->i
+(i+i)*i        T' -> $
+(i+i)*i        E' -> + T E'
(i+i)*i         T -> F T'
(i+i)*i         F -> ( E )
i+i)*i          E -> T E'
i+i)*i          T -> F T'
+i)*i           F ->i
+i)*i           T' -> $
+i)*i           E' -> + T E'
i)*i            T -> F T'
)*i             F ->i
)*i             T' -> $
)*i             E' -> $
*i              T' -> * F T'
                F ->i
                T' -> $
                 E' -> $
--------------------------------
String is successfully parsed
```

11. write a recursive descent parser for the grammar:
S□(L) S□a L□L,S L□S

Steps:

1. It repeatedly scans the ***input*** until we locate the right path.
2. The following qualities of the grammar must be present in order to create a recursive descent parser:
3. It shouldn't be allowed to repeat.
4. It needs to be ***left-factored***. Alternatives shouldn't share a prefix.
5. Recursion should be possible in languages.

6. Backtracking will be required by the recursive descent parser if the grammar is not left-factored.

Given Grammar:

Before removing left recursion	**After removing left recursion**
S□(L)|a L□L,S|S	S□(L) | a L□SL' L'□,SL | ε

Note: 'e' for epsilon.

```
#include <stdio.h>
#include <string.h>
#define SUCCESS 1
#define FAILED 0
int S(), L(), Ldash();
const char *cursor;
char string[64];
int main()
{
puts("Enter the string");
// scanf("%s", string);
sscanf("(a,a,a)", "%s", string);
cursor = string;
puts("");
puts("Input      Action");
puts("--------------------------------");
if (S() && *cursor == '\0') {
puts("--------------------------------");
puts("String is successfully parsed");
return 0;
} else {
puts("--------------------------------");
puts("Error in parsing String");
return 1;
}
}

int S()
{
printf("%-16s S□( L )'\n", cursor);
if (*cursor==’(‘) {
printf("%-16s S□( L )\n", cursor);
cursor++;
L();
If( *cursor==’)’)
{
Cursor++;
return SUCCESS;
}
else
```

```
return FAILED;
} else
If(*cursor==’a’)
{
Cursor++;
return SUCCESS;
}
}
int L()
{
Printf("%-16s L□SL’\n", cursor);
if (S()) {
if(Ldash()
return SUCCESS;
else
return FAILED;
}
else
return FAILED;
}

int Ldash()
{
if (*cursor == ',') {
printf("%-16s L' -> , S L’\n", cursor);
cursor++;
if (S()) {
if (Ldash())
return SUCCESS;
else
return FAILED;
} else
return FAILED;
} else {
printf("%-16s L' -> $\n", cursor);
return SUCCESS;
}
}
```

Output:

Input	Action
(a, a,a)	S□(L)
a,a,a	L→SL’
a,a,a	S→a
,a,a	L’→,SL’
a,a	S→a
,a	L’→,SL’
a	S→a
	L’→$

The string is successfully parsed.

12. write a C program to calculate First Function for the Grammar:
E□E+T E□T T□T*F T□F F□(E)|id
The rules for finding FIRST of a given grammar is:
If X is terminal, FIRST(X) = {X}.
If X → ε is a production, then add ε to FIRST(X).
If X is a non-terminal, and X → Y1 Y2 … Yk is a production, and ε is in all of FIRST(Y1), …, FIRST(Yk) then add ε to FIRST(X).
If X is a non-terminal, and X → Y1 Y2 … Yk is a production, then add a to FIRST(X) if for some i, a is in FIRST(Yi), and ε is in all of FIRST(Y1), …, FIRST(Yi-1).

Assumptions:

Each Non terminal character is represented by one Uppercase letter.
Each Terminal character is represented by one lowercase letter.
LHS and RHS of each production will be separated by a "=" symbol.
There will be no Blank space within a production string.
Assumed that Left recursion is avoided in all input productions.

```
#include<stdio.h>
#include<ctype.h>
void FIRST(char[],char );
void addToResultSet(char[],char);
int numOfProductions;
char productionSet[10][10];
int main()
{
   int i;
   char choice;
   char c;
   char result[20];
   printf("How many number of productions ? :");
   scanf(" %d",&numOfProductions);
   for(i=0;i<numOfProductions;i++)//read production string eg: E=E+T
   {
         printf("Enter productions Number %d : ",i+1);
         scanf(" %s",productionSet[i]);
   }
   do
   {
         printf("\n Find the FIRST of  :");
         scanf(" %c",&c);
         FIRST(result,c); //Compute FIRST; Get Answer in 'result' array
         printf("\n FIRST(%c)= { ",c);
         for(i=0;result[i]!='\0';i++)
         printf(" %c ",result[i]);       //Display result
         printf("}\n");
          printf("press 'y' to continue : ");
         scanf(" %c",&choice);
   }
   while(choice=='y'||choice =='Y');
   return 0;
}
```

```
/*
 *Function FIRST:
 *Compute the elements in FIRST(c) and write them
 *in Result Array.
 */
void FIRST(char* Result,char c)
{
   int i,j,k;
   char subResult[20];
   int foundEpsilon;
   subResult[0]='\0';
   Result[0]='\0';
   //If X is terminal, FIRST(X) = {X}.
   if(!(isupper(c)))
   {
      addToResultSet(Result,c);
            return ;
   }
   //If X is non terminal
   //Read each production
   for(i=0;i<numOfProductions;i++)
   {
//Find production with X as LHS
       if(productionSet[i][0]==c)
       {
//If X ? e is a production, then add e to FIRST(X).
 if(productionSet[i][2]=='$') addToResultSet(Result,'$');
          //If X is a non-terminal, and X ? Y1 Y2 … Yk
          //is a production, then add a to FIRST(X)
          //if for some i, a is in FIRST(Yi),
          //and e is in all of FIRST(Y1), …, FIRST(Yi-1).
     else
          {
             j=2;
             while(productionSet[i][j]!='\0')
             {
             foundEpsilon=0;
             FIRST(subResult,productionSet[i][j]);
             for(k=0;subResult[k]!='\0';k++)
                addToResultSet(Result,subResult[k]);
              for(k=0;subResult[k]!='\0';k++)
                 if(subResult[k]=='$')
                 {
                    foundEpsilon=1;
                    break;
                 }
              //No e found, no need to check next element
              if(!foundEpsilon)
                 break;
             j++;
             }
          }
   }
```

```
}
   return ;
}
/* addToResultSet adds the computed
 *element to result set.
 *This code avoids multiple inclusion of elements
  */
void addToResultSet(char Result[],char val)
{
   int k;
   for(k=0 ;Result[k]!='\0';k++)
      if(Result[k]==val)
         return;
   Result[k]=val;
   Result[k+1]='\0';
}
```

Input & Out put:

```
How many number of productions ?8
Enter the product 1: E=TA
Enter the product 2: A=+TA
Enter the product 3: A=#
Enter the product 4: T=FB
Enter the product 5: B=*FB
Enter the product 6: B=#
Enter the product 7: F=(E)
Enter the product 8: F=a
Find the First function of: E
FIRST(E):{ (,a }
Press ‘y’ to comtinue…:y
Find the First Fuction of:A
FIRST(A): { +, # }
Press ‘y’ to Continue..:n
```

13.write a yacc program to implement a top-down parser for the grammar:

Let's define a simple grammar for palindromes. We'll consider palindromes over the alphabet {a, b}. The grammar can be defined as follows:

S -> ε | a | b | aSa | bSb

Here's the Yacc code implementing the top-down parser for this palindrome grammar:

```
%{

#include <stdio.h>
#include <stdlib.h>
%}

%token a b
%left a b

%%
```

```
start : S '\n' { printf("Valid palindrome\n"); }
;

S    :  /* empty */   { }
|  a          { }
|  b          { }
|  a S a       { }
|  b S b       { }
;

%%

int main() {
printf("enter a string:");
yyparse();
return 0;
}

int yyerror(char *s) {
fprintf(stderr, "Error: %s\n", s);
return 0;
}

int yylex() {
int c = getchar();
if (c == EOF) return 0;
if (c == 'a') return a;
if (c == 'b') return b;
return c;
}
```

Compile: $ yacc -d exp_13.y

$ cc y.tab.c lex.yy.c

Execute: $ ./a.out

input:
enter a string: aabaa

out put:

it is a valid palindrome string.

14. write a yacc program to evaluate algebraic expression.

To evaluate Algebraic expression we use a Grammar:
T->T*T|T+T|T-T|T/T|-T|(T)|id

id: identifier, it may be number or variable. So it is token for this program. We return as NUMBER/ID.

Lex Code: evaluate_exp.l

```
%{
   /* Definition section*/
   #include "y.tab.h"
   extern yylval;
%}

%%
[0-9]+    {
            yylval = atoi(yytext);
            return NUMBER;
          }

[a-zA-Z]+    { return ID; }
[ \t]+        ;  /*For skipping whitespaces*/

\n            { return 0; }
.           { return yytext[0]; }

%%
```

Yacc Code: evaluate_exp.y

```
%{
   /* Definition section */
  #include <stdio.h>
%}

%token NUMBER ID
// setting the precedence
// and associativity of operators
%left '+' '-'
%left '*' '/'

/* Rule Section */
%%
E : T        {
               printf("Result = %d\n", $$);
               return 0;
          }

T :
   T '+' T { $$ = $1 + $3; }
   | T '-' T { $$ = $1 - $3; }
   | T '*' T { $$ = $1 * $3; }
   | T '/' T { $$ = $1 / $3; }
   | '-' NUMBER { $$ = -$2; }
   | '-' ID { $$ = -$2; }
   | '(' T ')' { $$ = $2; }
   | NUMBER { $$ = $1; }
```

```
    | ID {$$ = $1; };
% %

int main() {
   printf("Enter the expression\n");
   yyparse();
}

/* For printing error messages */
int yyerror(char* s) {
   printf("\nExpression is invalid\n");
}
```

OUT PUT:

```
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$ lex evaluate_exp.l
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$ yacc -d evaluate_exp.y
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$ cc lex.yy.c y.tab.c -ll
evaluate_exp.l:4:8: warning: type defaults to 'int' in declaration of 'yylval'
[-Wimplicit-int]
 extern yylval;
        ^~~~~~
y.tab.c: In function 'yyparse':
y.tab.c:1124:16: warning: implicit declaration of function 'yylex' [-Wimplicit-
function-declaration]
       yychar = yylex ();
                ^~~~~
y.tab.c:1314:7: warning: implicit declaration of function 'yyerror'; did you me
an 'yyerrok'? [-Wimplicit-function-declaration]
       yyerror (YY_("syntax error"));
       ^~~~~~~
       yyerrok
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$ ./a.out
Enter the expression
7*(5-3)/2
Result = 7
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$ ./a.out
Enter the expression
6/((3-2)*(-5+2))
Result = -2
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$
```

Viva-Voice:

1. Define algebraic expression
2. Example for algebraic expression